

Home Assignment Defect tracker and triage

Problem Statement

Use Case

Defect/Bug Triage

Operational Problem

In most engineering/QA organizations, defects are logged but not actively managed in real time:

- Bugs sit in a backlog untriaged for days with no assigned severity or priority.
- Critical/high-severity bugs breach their resolution SLA silently — nobody notices until a release is blocked.
- Assignment is manual and uneven — some developers get overloaded while others have spare capacity.
- Triage leads have no live view of what is new, what is aging, and what is about to breach SLA.
- Reopened bugs are treated the same as new ones, losing history and urgency context.
- Release/build managers cannot easily see which open defects are release-blocking.
- Managers only discover SLA breaches after the fact, during a status meeting or a release-readiness review.

AI Defect Triage & SLA Tracking Platform

1. Executive Summary

This document defines the target architecture and API contract for a real-time AI-assisted defect triage and SLA tracking platform.

The platform addresses:

- Untriaged defects remaining in backlog for days.
- Critical/high-severity defects silently approaching or breaching SLA.
- Manual defect assignment.
- Lack of live triage and SLA visibility.

- Reopened defects losing lifecycle context.
- Lack of explicit release-blocking visibility.
- Late discovery of SLA breaches.

Technology

Layer	Technology

Frontend	React.js
Backend	Node.js + TypeScript
Operational DB	MongoDB
Vector Search	MongoDB Vector Search
Embeddings	OpenAI
LLM	OpenAI
Real-time	WebSocket
External Defect Systems	Jira / Azure DevOps
Code Repository	GitHub / Bitbucket API/Search
Cloud	AWS
Compute	ECS/Fargate

The AI is used as a decision-support layer. Severity and priority are recommended by AI and approved or overridden by the Triage Lead.

2. Questionnaire Decisions

#	Decision	Selected

1	Bug intake	React UI + Jira/ADO integration
2	Jira/ADO synchronization	Two-way synchronization
3	Severity/Priority	AI recommendation + Triage Lead approval
4	SLA calculation	Severity + Priority
5	SLA start	After Triage Lead approval
6	SLA calendar	Business hours
7	At-risk threshold	80% of SLA consumed
8	SLA action	Notification only
9	Developer assignment	Manual assignment
10	Reopened defect	Treat as new defect / new triage cycle
11	Release blocking	Manually marked by Triage Lead
12	AI knowledge sources	All specified vectorized sources

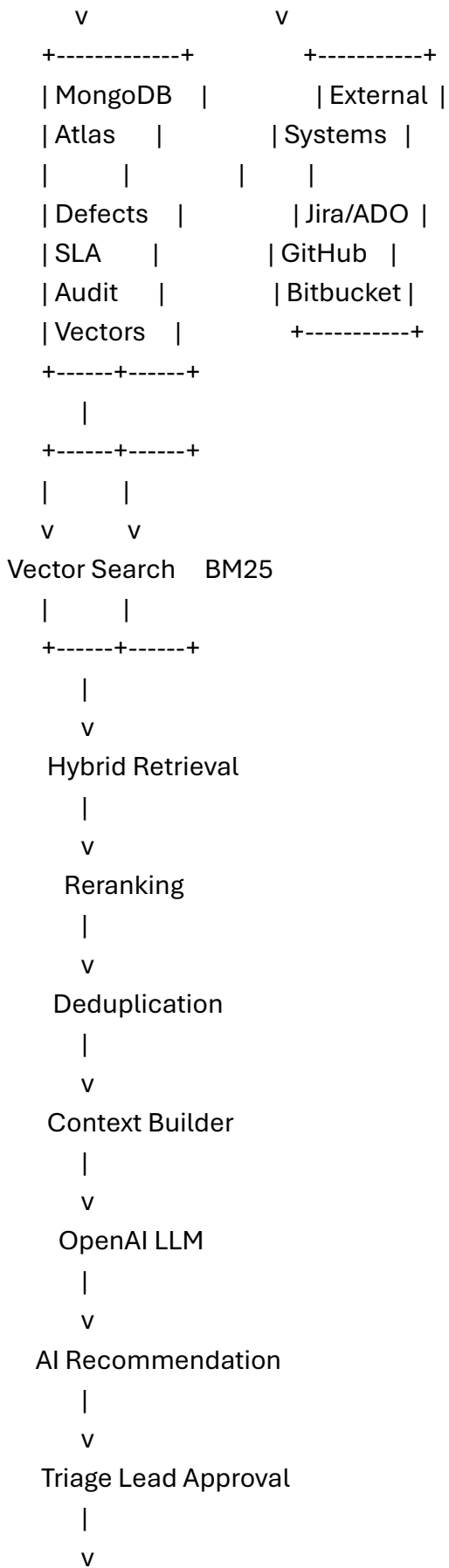
- | | | |
|----|----------------|---|
| 13 | Developer code | Repository API/search; no vectorization |
| 14 | Retrieval | Hybrid Vector + BM25 + reranking |
| 15 | Embeddings | OpenAI only |
| 16 | LLM | OpenAI only |
| 17 | Database | MongoDB operational + vector |
| 18 | Real-time UI | WebSocket |
| 19 | Authorization | Jira/ADO permissions |
| 20 | AWS deployment | ECS/Fargate + MongoDB Atlas |

3. High-Level Architecture

```

graph TD
    USERS --> V1((v))
    V1 --- L1[+-----+]
    L1 --- UI[| React.js UI |]
    UI --- L2[+-----+-----+]
    L2 --- V2((v))
    V2 --- HTTPS[HTTPS/WebSocket]
    HTTPS --- V3((v))
    V3 --- L3[+-----+]
    L3 --- NTS[| Node.js / TypeScript |]
    NTS --- L4[| |]
    L4 --- DM[| Defect Management |]
    DM --- TRI[| Triage |]
    TRI --- SLA[| SLA |]
    SLA --- ASS[| Assignment |]
    ASS --- REL[| Release |]
    REL --- INT[| Integration |]
    INT --- ING[| Ingestion |]
    ING --- RET[| Retrieval |]
    RET --- AI[| AI |]
    AI --- NOT[| Notification |]
    NOT --- AUD[| Audit |]
    AUD --- L5[+-----+-----+]
    L5 --- L6[| |]
    L6 --- L7[+-----+-----+]
    L7 --- L8[| |]

```



Authoritative State

```

---

## 4. API Design Principles

### 4.1 Base URL

The logical API base path is:

``` text

/api/v1

```

Production example:

``` text

https://<platform-domain>/api/v1

```

### 4.2 Common Headers

``` http

Authorization: Bearer <token>

Content-Type: application/json

X-Correlation-Id: <uuid>

```

For external integration requests:

``` http

X-Webhook-Signature: <signature>

X-Source-System: jira|ado

```

### 4.3 API Response Convention

Successful response:

``` json

```
{  
  "success": true,  
  "data": {},  
  "requestId": "req-123"  
}
```

```

Error response:

```

``` json
{
  "success": false,
  "error": {
    "code": "DEFECT\_NOT\_FOUND",
    "message": "Defect was not found"
  },
  "requestId": "req-123"
}
```

```

## 4.4 HTTP Status Codes

| Status | Usage |
|--------|-------|
|--------|-------|

|         |                                       |
|---------|---------------------------------------|
| 200     | Successful read/update                |
| 201     | Resource created                      |
| 202     | Async operation accepted              |
| 204     | Successful operation with no body     |
| 400     | Validation error                      |
| 401     | Authentication failure                |
| 403     | Authorization failure                 |
| 404     | Resource not found                    |
| 409     | Conflict/idempotency/version conflict |
| 422     | Business validation failure           |
| 429     | Rate limit                            |
| 500     | Internal server error                 |
| 502/503 | External dependency unavailable       |

## 5. API Endpoint Inventory

### 5.1 Authentication / Current User

Because authorization is based on Jira/ADO permissions, the platform should expose only minimal identity/session information.

| Method | Endpoint | Purpose |
|--------|----------|---------|
|--------|----------|---------|

GET `/me`` Return current  
authenticated user's  
identity and source  
permissions

GET `/me/permissions`` Return effective  
Jira/ADO permissions  
relevant to the  
platform

GET `/me``

Example response:

```
```json
{
  "success": true,
  "data": {
    "userId": "u123",
    "displayName": "John Doe",
    "email": "user@example.com",
    "sourceSystem": "jira",
    "permissions": [
      "DEFECT_CREATE",
      "TRIAGE_APPROVE"
    ]
  }
}
```
```

---

## 6. Defect Management APIs

These APIs manage the core defect lifecycle.

### 6.1 Create Defect

```
```http
POST /api/v1/defects
```
```

#### Purpose

Create a defect from the React application.

#### Request

```
``` json
{
  "title": "Payment API returns incorrect status",
  "description": "Payment is successful but response shows failed.",
  "product": "Payments",
  "module": "Payment API",
  "environment": "QA",
  "release": "R2026.10",
  "build": "2026.10.15",
  "reproductionSteps": \[
    "Create payment",
    "Submit payment",
    "Check response"
  ],
  "expectedResult": "Payment should return SUCCESS",
  "actualResult": "Response returns FAILED",
  "attachments": \[]
}
```
```

## Response

```
``` json
{
  "success": true,
  "data": {
    "defectId": "DEF-1001",
    "status": "NEW",
    "triageStatus": "PENDING"
  }
}
```
```

---

## 6.2 List Defects

```
``` http
GET /api/v1/defects
```
```

## Query parameters

```
``` text
status
triageStatus
severity
priority
assignee
release
build
slaStatus
releaseBlocking
sourceSystem
page
pageSize
sortBy
sortOrder
```
```

Example:

```
``` http
GET /api/v1/defects?severity=CRITICAL&slaStatus=AT_RISK&release=R2026.10
```
```

---

### 6.3 Get Defect

```
``` http
GET /api/v1/defects/{defectId}
```
```

Returns:

- Defect details.
- Current lifecycle status.
- Severity/Priority.
- SLA status.
- Assignment.
- Release-blocking status.
- Current triage recommendation.
- Source-system references.

---

#### 6.4 Update Defect

```
``` http
PATCH /api/v1/defects/{defectId}
```
```

Used for permitted editable fields.

Example:

```
``` json
{
  "description": "Updated reproduction information",
  "environment": "UAT"
}
```
```

---

#### 6.5 Add Defect Comment

```
``` http
POST /api/v1/defects/{defectId}/comments
```
```

Request:

```
``` json
{
  "comment": "Developer identified the affected payment service."
}
```
```

---

#### 6.6 Get Defect Comments

```
``` http
GET /api/v1/defects/{defectId}/comments
```
```

---

#### 6.7 Change Defect Status

```
``` http
POST /api/v1/defects/{defectId}/status
```
```

Request:

```
``` json
{
  "status": "IN_PROGRESS",
  "reason": "Developer started investigation"
}
```
```

Allowed transitions should be validated by the lifecycle service.

---

## 7. Triage APIs

### 7.1 Generate AI Triage Recommendation

```
``` http
POST /api/v1/defects/{defectId}/triage/recommend
```
```

#### Purpose

Starts the AI-assisted triage workflow.

#### Processing

```
``` text
```

Defect

|

v

Query Construction

|

v

Normalization

|

v

Vector Search + BM25

|

v

Hybrid Retrieval

|

v

Reranking

|

v

Deduplication

|

v

Context Building

|

v

Optional Code Search

|

v

OpenAI

|

v

Recommendation

```

Response

``` json

```
{
  "success": true,
  "data": {
    "triageRecommendationId": "TR-9001",
    "severity": "HIGH",
    "priority": "P1",
    "summary": "Issue affects successful payment processing.",
    "evidence": \[
      {
        "sourceType": "REQUIREMENT",
        "sourceId": "US-123"
      },
      {
        "sourceType": "TEST_CASE",
        "sourceId": "TC-456"
      }
    ],
    "status": "AWAITING_APPROVAL"
  }
}
```

```

This endpoint must not directly update authoritative Severity/Priority.

---

## 7.2 Get Triage Recommendation

```
``` http
GET /api/v1/defects/{defectId}/triage/recommendation
```
```

Returns the latest AI recommendation and supporting evidence.

---

## 7.3 Approve AI Recommendation

```
``` http
POST /api/v1/defects/{defectId}/triage/approve
```
```

Request:

```
``` json
{
  "recommendationId": "TR-9001",
  "comment": "Approved based on production-impacting payment behavior."
}
```
```

Processing

Approval triggers:

```
``` text
Severity/Priority Approved
  |
  v
SLA Policy Lookup
  |
  v
Business Calendar Calculation
  |
  v
SLA Start
```
```

---

## 7.4 Override AI Recommendation

```
``` http
POST /api/v1/defects/{defectId}/triage/override
```
```

Request:

```
``` json
{
  "severity": "MEDIUM",
  "priority": "P2",
  "reason": "Impact is limited to a non-production environment."
}
```
```

The override must be audited.

---

## 7.5 Get Triage History

```
``` http
GET /api/v1/defects/{defectId}/triage/history
```
```

Returns:

1. AI recommendations.
2. Approval actions.
3. Overrides.
4. Actor.
5. Timestamp.
6. Reason.
7. Recommendation evidence.

---

## 8. SLA APIs

### 8.1 Get Defect SLA

```
``` http
GET /api/v1/defects/{defectId}/sla
```
```

Example:

```
``` json
{
  "success": true,
  "data": {
    "status": "AT_RISK",
    "severity": "HIGH",
    "priority": "P1",
    "slaStartedAt": "2026-10-03T10:00:00Z",
    "slaDueAt": "2026-10-05T10:00:00Z",
    "consumedPercentage": 84,
    "remainingBusinessMinutes": 480
  }
}
```
```

---

## 8.2 SLA Dashboard

```
``` http
GET /api/v1/sla/dashboard
```
```

Query parameters:

```
``` text
release
severity
priority
assignee
team
dateFrom
dateTo
```
```

Returns aggregate counts:

```
``` json
{
  "onTrack": 42,
  "atRisk": 8,
  "breached": 3
}
```

```
}  
` ``
```

8.3 At-Risk Defects

```
` `` http  
GET /api/v1/sla/at-risk  
` ``
```

Returns defects at or above 80% SLA consumption.

8.4 Breached Defects

```
` `` http  
GET /api/v1/sla/breached  
` ``
```

Returns defects whose SLA has reached 100%.

8.5 SLA Policies

The initial requirement defines SLA based on Severity + Priority.

List policies

```
` `` http  
GET /api/v1/sla/policies  
` ``
```

Get policy

```
` `` http  
GET /api/v1/sla/policies/{policyId}  
` ``
```

Create policy

```
` `` http  
POST /api/v1/sla/policies  
` ``
```

Update policy

```
``` http
PATCH /api/v1/sla/policies/{policyId}
```
```

Example:

```
``` json
{
 "severity": "HIGH",
 "priority": "P1",
 "durationBusinessMinutes": 480,
 "businessCalendarId": "DEFAULT"
}
```
```

These policy-management endpoints should be restricted to authorized administrative users according to the connected Jira/ADO permission model.

9. Assignment APIs

Assignment is manual.

9.1 Assign Defect

```
``` http
POST /api/v1/defects/{defectId}/assignment
```
```

Request:

```
``` json
{
 "assigneeId": "dev-123",
 "reason": "Payment API ownership"
}
```
```

9.2 Reassign Defect

```
``` http
POST /api/v1/defects/{defectId}/assignment/reassign
```
```

Request:

```
``` json
{
 "assigneeId": "dev-456",
 "reason": "Original developer unavailable"
}
```
```

9.3 Get Assignment

```
``` http
GET /api/v1/defects/{defectId}/assignment
```
```

9.4 Assignment History

```
``` http
GET /api/v1/defects/{defectId}/assignment/history
```
```

9.5 Developer Workload View

Although assignment is manual, workload visibility is required.

```
``` http
GET /api/v1/workload/developers
```
```

Query parameters:

```
``` text
team
module
release
includeClosed
```
```

Example response:

```
``` json
{
 "developers": \[
```

```
{
 "developerId": "dev-123",
 "openDefects": 8,
 "critical": 1,
 "high": 3,
 "atRisk": 2,
 "breached": 0
}
]
```

The endpoint provides awareness only. It does not automatically assign defects.

---

## 10. Reopen APIs

### 10.1 Reopen Defect

```
``` http
POST /api/v1/defects/{defectId}/reopen
```
```

Request:

```
``` json
{
  "reason": "Issue reproduced after deployment."
}
```
```

Processing

```
``` text
Closed Defect
```

```
|
v
```

Reopened

```
|
v
```

New Triage Cycle

```
|
+--> New AI recommendation
+--> New approval
```

```
+--> New SLA
+--> New assignment
` ` `
```

10.2 Get Triage Cycles

```
` ` ` http
GET /api/v1/defects/{defectId}/triage-cycles
` ` `
```

Returns all historical triage cycles.

10.3 Get Specific Triage Cycle

```
` ` ` http
GET /api/v1/defects/{defectId}/triage-cycles/{cycleId}
` ` `
```

11. Release APIs

11.1 Mark Release Blocking

```
` ` ` http
POST /api/v1/defects/{defectId}/release-block
` ` `
```

Request:

```
` ` ` json
{
  "releaseId": "REL-2026.10",
  "reason": "Payment processing release cannot proceed with this defect."
}
` ` `
```

11.2 Remove Release Blocking

```
` ` ` http
DELETE /api/v1/defects/{defectId}/release-block
` ` `
```

Request:

```
``` json
{
 "reason": "Defect fixed and verified."
}
```
```

11.3 Get Release-Blocking Defects

```
``` http
GET /api/v1/releases/{releaseId}/defects/release-blocking
```
```

11.4 Get Release Defect Dashboard

```
``` http
GET /api/v1/releases/{releaseId}/defects
```
```

Query parameters:

```
``` text
severity
priority
slaStatus
status
releaseBlocking
```
```

12. Release / Build APIs

12.1 List Releases

```
``` http
GET /api/v1/releases
```
```

12.2 Get Release

```
``` http
GET /api/v1/releases/{releaseId}
```
```

12.3 List Release Defects

```
``` http
GET /api/v1/releases/{releaseId}/defects
```
```

12.4 List Release Builds

```
``` http
GET /api/v1/releases/{releaseId}/builds
```
```

12.5 Get Build Defects

```
``` http
GET /api/v1/releases/{releaseId}/builds/{buildId}/defects
```
```

13. Retrieval APIs

The retrieval APIs are primarily backend/service APIs rather than public end-user APIs.

13.1 Search Knowledge

```
``` http
POST /api/v1/knowledge/search
```
```

Request:

```
``` json
{
 "query": "payment API failed transaction",
 "filters": {
 "product": "Payments",
 "release": "R2026.10"
 },
 "topK": 20
}
```
```

Processing:

```
``` text
Query
|
+--> Normalization
```

+--> Abbreviation Expansion

+--> Synonym Expansion

|

+--> Vector Search

|

+--> BM25

|

v

Hybrid Search

|

v

Reranking

|

v

Deduplication

```

13.2 Get Knowledge Source

``` http

GET /api/v1/knowledge/{knowledgeId}

```

13.3 Retrieve Similar Defects

``` http

GET /api/v1/knowledge/similar-defects

```

Query:

``` text

defectId

topK

```

14. Ingestion APIs

14.1 Start Ingestion Job

```
``` http
POST /api/v1/ingestion/jobs
```
```

Request:

```
``` json
{
 "sourceType": "CONFLUENCE",
 "projectId": "PAYMENTS",
 "fullSync": false
}
```
```

Supported source types include:

```
``` text
REQUIREMENT
FIGMA
SWAGGER
TEST_CASE
TEST_DATA
JIRA
ADO
TESTRAIL
ZEPHYR
XRAY
MEETING_NOTE
MEETING_RECORDING
RELEASE_NOTE
CONFLUENCE
WIKI
TECHNICAL_DOCUMENTATION
DEFECT_DATABASE
```
```

14.2 Get Ingestion Job

```
``` http
GET /api/v1/ingestion/jobs/{jobId}
```
```

14.3 List Ingestion Jobs

```
``` http
GET /api/v1/ingestion/jobs
```
```

Query parameters:

```
``` text
sourceType
status
dateFrom
dateTo
```
```

14.4 Retry Failed Ingestion

```
``` http
POST /api/v1/ingestion/jobs/{jobId}/retry
```
```

14.5 Get Ingestion Statistics

```
``` http
GET /api/v1/ingestion/statistics
```
```

Example:

```
``` json
{
 "documentsProcessed": 15000,
 "chunksCreated": 62000,
 "failedDocuments": 12,
 "lastSuccessfulSync": "2026-10-03T08:00:00Z"
}
```
```

15. Developer Code Search APIs

Developer code is not vectorized.

15.1 Search Repository

```
``` http
POST /api/v1/code/search
```
```

Request:

```
``` json
{
 "repository": "payments-service",
 "query": "payment status response mapping",
 "branch": "main",
 "topK": 10
}
```
```

The service searches GitHub/Bitbucket through repository APIs/search capabilities.

15.2 Get Code Context

```
``` http
GET /api/v1/code/context
```
```

Query:

```
``` text
repository
filePath
startLine
endLine
commit
```
```

The resulting code context can be passed to the AI triage service when required.

16. Jira / ADO Integration APIs

The integration layer supports two-way synchronization.

16.1 Jira Webhook

```
``` http
POST /api/v1/integrations/jira/webhook
```
```

Receives Jira defect changes.

16.2 ADO Webhook

```
``` http
POST /api/v1/integrations/ado/webhook
```
```

Receives Azure DevOps work-item changes.

16.3 Pull Jira/ADO Changes

```
``` http
POST /api/v1/integrations/{sourceSystem}/sync
```
```

Example:

```
``` http
POST /api/v1/integrations/jira/sync
```
```

Request:

```
``` json
{
 "projectId": "PAYMENTS",
 "since": "2026-10-02T00:00:00Z"
}
```
```

16.4 Get Integration Status

```
``` http
GET /api/v1/integrations/{sourceSystem}/status
```
```

16.5 Get Sync History

```
``` http
GET /api/v1/integrations/{sourceSystem}/sync-history
```
```

16.6 Retry Synchronization

```
``` http
POST /api/v1/integrations/{sourceSystem}/sync/{syncId}/retry
```
```

17. Notification APIs

The selected business behavior is notification only.

17.1 Get Notifications

```
``` http
GET /api/v1/notifications
```
```

Query:

```
``` text
unread
type
dateFrom
dateTo
```
```

17.2 Mark Notification Read

```
``` http
POST /api/v1/notifications/{notificationId}/read
```
```

17.3 Get Notification History

```
``` http
GET /api/v1/defects/{defectId}/notifications
```
```

Notification types:

```
``` text
SLA\AT\RISK
SLA\BREACHED
TRIAGE\PENDING
ASSIGNMENT\CHANGED
REOPENED
RELEASE\BLOCK\CHANGED
```
```

18. Dashboard APIs

18.1 Triage Dashboard

```
``` http
GET /api/v1/dashboard/triage
```
```

Returns:

- New defects.
- Untriaged defects.
- AI recommendations awaiting approval.
- On Track.
- At Risk.
- Breached.
- Reopened.
- Release-blocking.

18.2 SLA Dashboard

```
``` http
GET /api/v1/dashboard/sla
```
```

18.3 Developer Dashboard

```
``` http
GET /api/v1/dashboard/developer
```
```

18.4 Release Dashboard

```
``` http
GET /api/v1/dashboard/release/{releaseId}
```
```

19. Audit APIs

Every authoritative change should be auditable.

19.1 Defect Audit

```
``` http
GET /api/v1/defects/{defectId}/audit
```
```

19.2 Entity Audit

```
``` http
GET /api/v1/audit/{entityType}/{entityId}
```
```

Audit examples:

```
``` text
SEVERITY_CHANGED
PRIORITY_CHANGED
TRIAGE_APPROVED
TRIAGE_OVERRIDDEN
SLA_STARTED
SLA_STATUS_CHANGED
ASSIGNEE_CHANGED
RELEASE_BLOCKED
RELEASE_BLOCK_REMOVED
STATUS_CHANGED
REOPENED
CLOSED
SYNCED
```
```

20. WebSocket Events

The application uses WebSocket for real-time UI updates.

Connection

Logical endpoint:

```
```text
wss://<platform-domain>/ws
```
```

Events

```
```text
DEFECT_CREATED
DEFECT_UPDATED
TRIAGE_RECOMMENDATION_READY
TRIAGE_APPROVED
TRIAGE_OVERRIDDEN
SLA_STARTED
SLA_AT_RISK
SLA_BREACHED
ASSIGNMENT_CHANGED
DEFECT_REOPENED
DEFECT_CLOSED
RELEASE_BLOCKED
RELEASE_BLOCK_REMOVED
SYNC_COMPLETED
SYNC_FAILED
```
```

Example:

```
```json
{
 "event": "SLA_AT_RISK",
 "timestamp": "2026-10-03T12:00:00Z",
 "data": {
 "defectId": "DEF-1001",
 "slaStatus": "AT_RISK",
 "consumedPercentage": 80
 }
}
```

```
}
` ``
```

---

## 21. Complete Endpoint Matrix

---

Functionality	Method	Endpoint
Current user	GET	`/me`
Permissions	GET	`/me/permissions`
Create defect	POST	`/defects`
List defects	GET	`/defects`
Get defect	GET	`/defects/{id}`
Update defect	PATCH	`/defects/{id}`
Comments	POST	`/defects/{id}/comments`
Get comments	GET	`/defects/{id}/comments`
Change status	POST	`/defects/{id}/status`
AI triage	POST	`/defects/{id}/triage/recommend`
Get recommendation	GET	`/defects/{id}/triage/recommendation`
Approve triage	POST	`/defects/{id}/triage/approve`
Override triage	POST	`/defects/{id}/triage/override`
Triage history	GET	`/defects/{id}/triage/history`
Get SLA	GET	`/defects/{id}/sla`
SLA dashboard	GET	`/sla/dashboard`
At-risk defects	GET	`/sla/at-risk`
Breached defects	GET	`/sla/breached`
List SLA policies	GET	`/sla/policies`
Create SLA policy	POST	`/sla/policies`
Update SLA policy	PATCH	`/sla/policies/{id}`

Assign	POST	`/defects/{id}/assignment`
Reassign	POST	`/defects/{id}/assignment/reassign`
Get assignment	GET	`/defects/{id}/assignment`
Assignment history	GET	`/defects/{id}/assignment/history`
Developer workload	GET	`/workload/developers`
Reopen	POST	`/defects/{id}/reopen`
Triage cycles	GET	`/defects/{id}/triage-cycles`
Triage cycle detail	GET	`/defects/{id}/triage-cycles/{cycleId}`
Mark release blocking	POST	`/defects/{id}/release-block`
Remove release blocking	DELETE	`/defects/{id}/release-block`
Release-blocking defects	GET	`/releases/{id}/defects/release-blocking`
Release defects	GET	`/releases/{id}/defects`
Releases	GET	`/releases`
Release detail	GET	`/releases/{id}`
Release builds	GET	`/releases/{id}/builds`
Build defects	GET	`/releases/{id}/builds/{buildId}/defects`
Knowledge search	POST	`/knowledge/search`
Knowledge item	GET	`/knowledge/{id}`
Similar defects	GET	`/knowledge/similar-defects`
Start ingestion	POST	`/ingestion/jobs`
Ingestion job	GET	`/ingestion/jobs/{id}`
Ingestion jobs	GET	`/ingestion/jobs`
Retry ingestion	POST	`/ingestion/jobs/{id}/retry`
Ingestion statistics	GET	`/ingestion/statistics`
Code search	POST	`/code/search`
Code context	GET	`/code/context`
Jira webhook	POST	`/integrations/jira/webhook`

ADO webhook	POST	`/integrations/ado/webhook`
External sync	POST	`/integrations/{source}/sync`
Integration status	GET	`/integrations/{source}/status`
Sync history	GET	`/integrations/{source}/sync-history`
Retry sync	POST	`/integrations/{source}/sync/{id}/retry`
Notifications	GET	`/notifications`
Mark notification read	POST	`/notifications/{id}/read`
Defect notifications	GET	`/defects/{id}/notifications`
Triage dashboard	GET	`/dashboard/triage`
SLA dashboard	GET	`/dashboard/sla`
Developer dashboard	GET	`/dashboard/developer`
Release dashboard	GET	`/dashboard/release/{id}`
Defect audit	GET	`/defects/{id}/audit`
Entity audit	GET	`/audit/{entityType}/{entityId}`
WebSocket	WS	`/ws`

---

## 22. API-to-Service Mapping

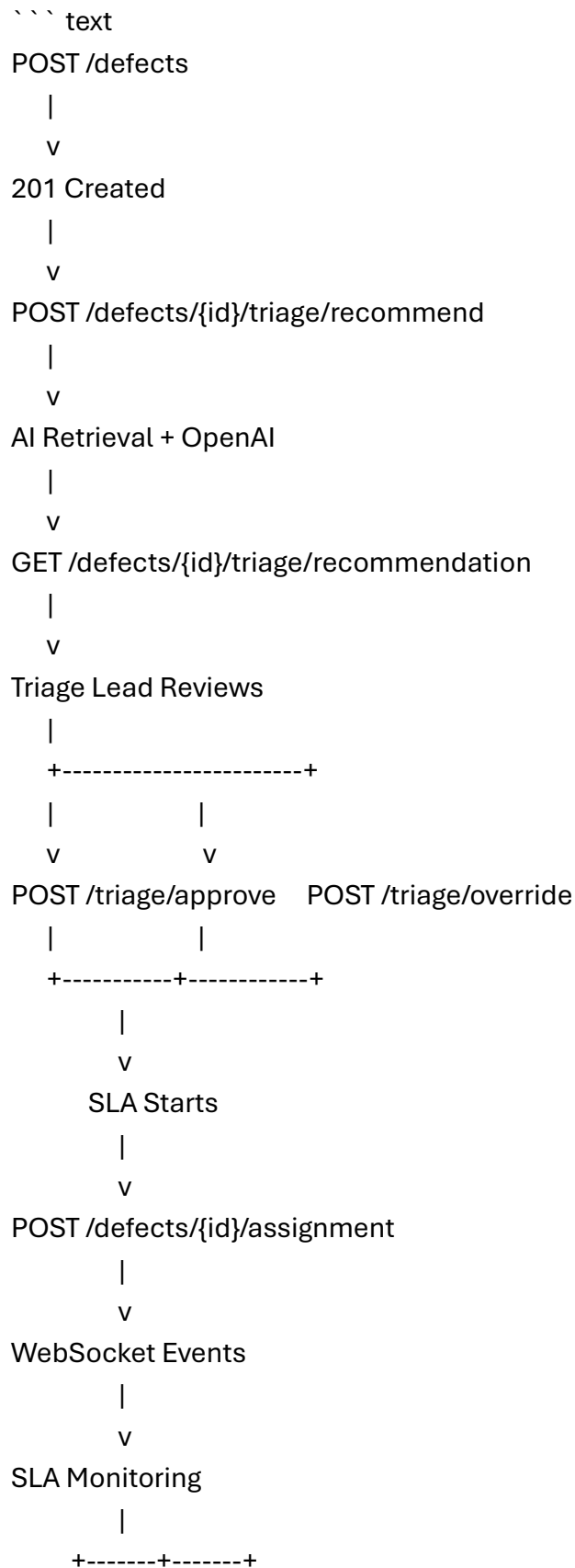
API Area	Backend Service
----------	-----------------

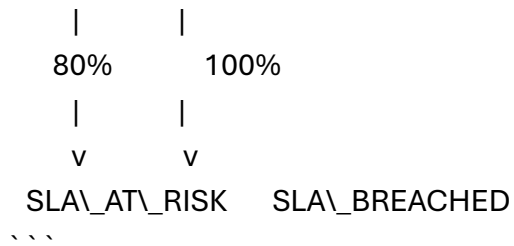
---

`/defects`	Defect Service
`/triage`	AI Triage Service
`/sla`	SLA Service
`/assignment`	Assignment Service
`/release`	Release Service
`/knowledge`	Retrieval Service
`/ingestion`	Ingestion Service
`/code`	Code Search Service
`/integrations`	Integration Service
`/notifications`	Notification Service
`/dashboard`	Dashboard/Query Service
`/audit`	Audit Service
`/ws`	WebSocket Service

---

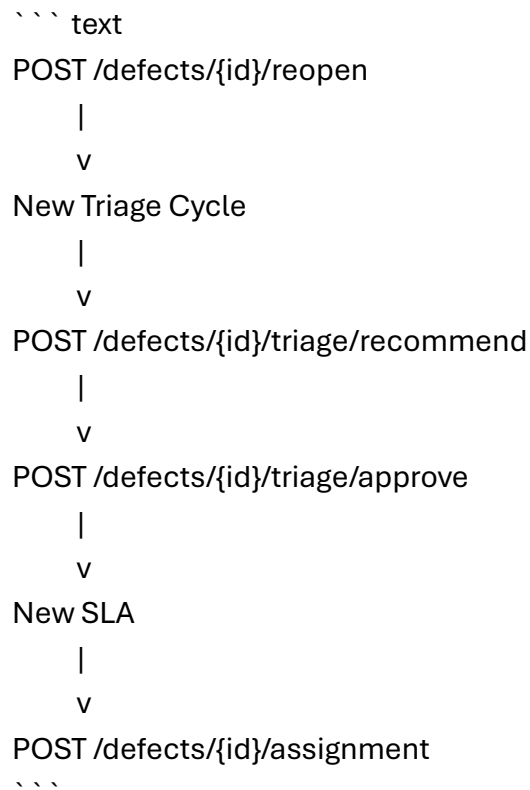
### 23. End-to-End API Sequence: New Defect



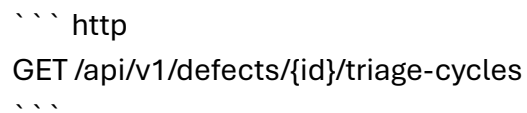


---

## 24. End-to-End API Sequence: Reopened Defect

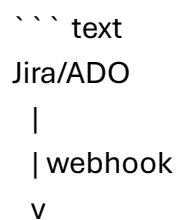


Historical triage cycles remain accessible through:

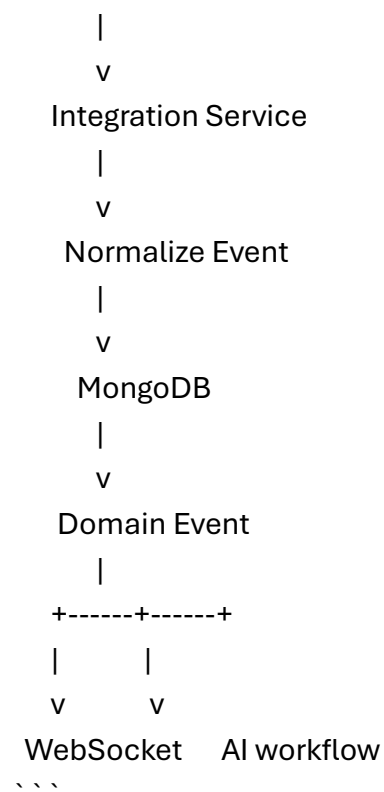


---

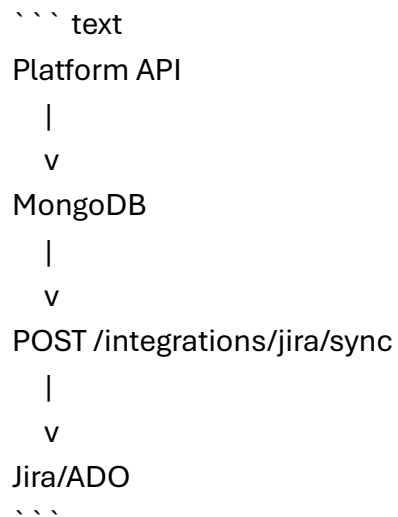
## 25. End-to-End API Sequence: Jira/ADO Synchronization



POST /integrations/jira/webhook



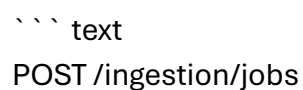
For platform-originated changes:



The integration service must use idempotency and origin tracking to prevent synchronization loops.

---

## 26. End-to-End API Sequence: Knowledge Ingestion



```

 |
 v
Connector
 |
 v
Extract
 |
 v
Normalize
 |
 v
Chunk
 |
 v
OpenAI Embedding
 |
 v
MongoDB
 |
 v
Vector Index
` ` `

```

Monitoring:

```

` ` ` text
GET /ingestion/jobs/{id}
GET /ingestion/statistics
` ` `

```

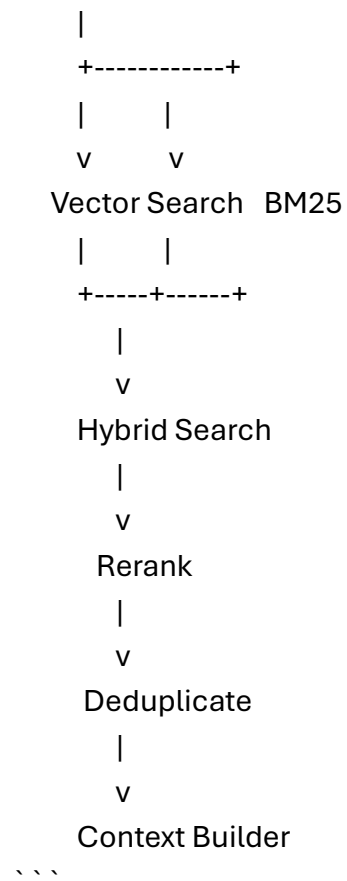
---

## 27. End-to-End API Sequence: AI Retrieval

```

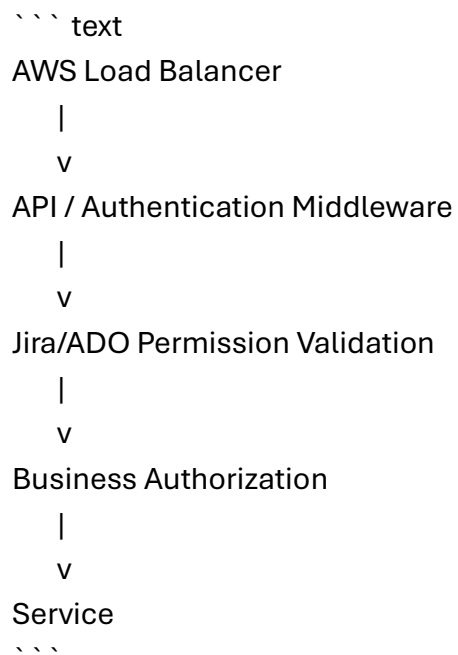
` ` ` text
POST /knowledge/search
 |
 v
Query preprocessing
 |
 +--> Normalization
 +--> Abbreviation expansion
 +--> Synonym expansion

```



## 28. API Security

Every application API should pass through:



Sensitive endpoints include:

8. Triage approval.
9. Triage override.
10. SLA policy administration.
11. Assignment.
12. Release-blocking.
13. Integration configuration.
14. Ingestion administration.

These must validate the caller's effective Jira/ADO permissions.

Webhook endpoints must validate their source-system signature/authentication before processing.

---

## 29. API Idempotency

Idempotency is required for operations that may be retried.

Recommended header:

```
``` http
Idempotency-Key: <unique-key>
```
```

Especially for:

```
``` text
POST /defects
POST /defects/{id}/triage/recommend
POST /defects/{id}/triage/approve
POST /defects/{id}/assignment
POST /defects/{id}/reopen
POST /integrations/{source}/sync
```
```

Webhook events should also use an external event ID.

---

## 30. API Validation Rules

Examples:

## Defect

- Title is mandatory.
- Description is mandatory.
- Source system must be valid.
- Release must exist when supplied.

## Triage Approval

- Recommendation must exist.
- Recommendation must belong to the defect.
- Recommendation must be in `AWAITING\_APPROVAL`.
- Severity and priority must be valid.

## SLA

- SLA can start only after approved Severity + Priority.
- SLA must use the configured business calendar.
- SLA cannot start twice for the same triage cycle.

## Reopen

- Closed/resolved defect only.
- New triage cycle must be created.
- Previous lifecycle must remain immutable.

## Release Blocking

- Defect must belong to the specified release.
- Actor must have appropriate permission.
- Action must be audited.

---

## 31. Core MongoDB Collections

```text

defects

defect_lifecycle

triage_recommendations

triage_cycles

sla_policies
sla_instances
knowledge_chunks
ingestion_jobs
assignments
notifications
audit_logs
sync_records
releases
builds
` ` `

32. Critical Functional Modules

Critical

1. Real-time defect creation.
2. Jira/ADO two-way synchronization.
3. AI Severity/Priority recommendation.
4. Triage Lead approval/override.
5. SLA calculation.
6. Business-hours SLA tracking.
7. 80% At-Risk state.
8. Breach detection.
9. Notifications.
10. Manual assignment.
11. Reopen/new triage cycle.
12. Manual Release Blocking.
13. Ingestion pipeline.
14. Hybrid retrieval.
15. OpenAI embeddings.
16. OpenAI LLM.
17. MongoDB Vector Search.

18. BM25.
19. Reranking.
20. WebSocket dashboard.
21. Audit history.

33. Explicitly Out of Scope

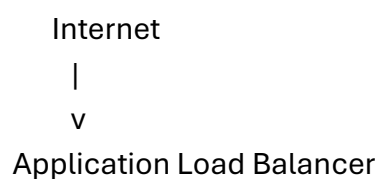
Based on the questionnaire answers:

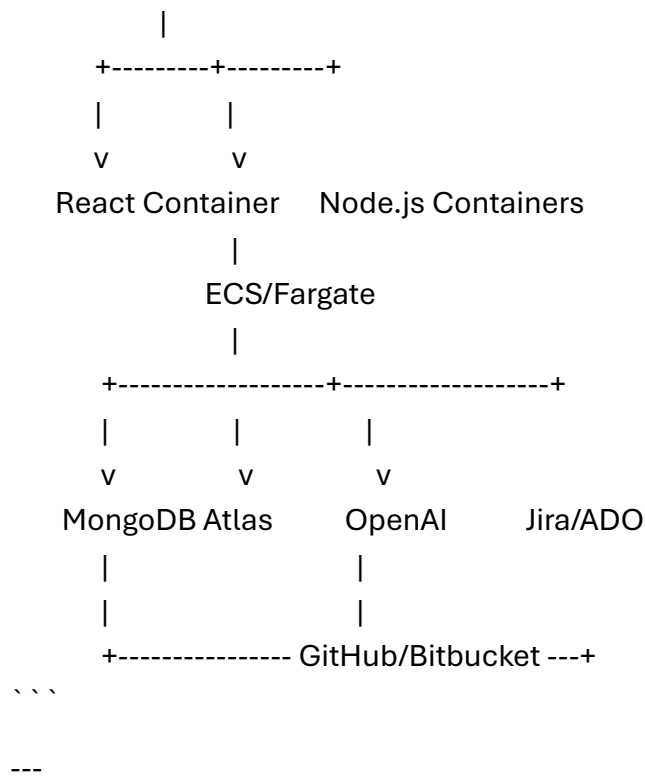
- Slack/Teams/email defect intake.
- Public defect-creation API as a user intake channel.
- Automatic developer assignment.
- AI automatic assignment.
- Automatic escalation.
- Automatic reassignment.
- AI automatic release-blocking decision.
- Groq LLM.
- Mistral embeddings.
- Code vectorization.
- Separate relational database.
- Separate RBAC/identity layer.
- Kubernetes/EKS.

The REST APIs in this document are the **platform's internal/application APIs**, not an additional external defect-intake channel.

34. AWS Deployment

``` text





### 35. Observability

API metrics should include:

```

\ \ \ text
request count
error count
latency
status code
endpoint
correlation ID
external dependency latency
\ \ \

```

AI metrics:

```

\ \ \ text
retrieval latency
embedding latency
reranking latency
LLM latency
token usage
AI recommendation count
AI override count
\ \ \

```

Business metrics:

```
``` text
untriaged defects
average triage time
at-risk defects
breached defects
release-blocking defects
reopened defects
developer workload
```
```

---

### 36. Failure Handling

OpenAI unavailable

Defect creation must still succeed.

```
``` text
AI\_TRiage\_PENDING
```
```

The Triage Lead can manually classify the defect.

Jira/ADO unavailable

Maintain:

```
``` text
SYNC\_PENDING
```
```

and retry according to the integration strategy.

GitHub/Bitbucket unavailable

AI triage can continue without code context, with missing code context explicitly recorded.

MongoDB unavailable

Do not report a successful persistent transaction when MongoDB has not confirmed the write.

---

### 37. Final Architecture

The target processing model is:

```
``` text
INGEST
|
v
NORMALIZE
|
v
CHUNK
|
v
OPENAI EMBEDDING
|
v
MONGODB
|
v
VECTOR + BM25 RETRIEVAL
|
v
RERANK
|
v
DEDUPLICATE
|
v
CONTEXT BUILDING
|
v
OPENAI LLM
|
v
AI SEVERITY + PRIORITY RECOMMENDATION
|
v
TRIAGE LEAD APPROVAL
|
v
SLA START
|
```

v
MANUAL ASSIGNMENT
|
v
REAL-TIME MONITORING
|
+---- 80% ----> NOTIFICATION
|
+---- 100% ---> NOTIFICATION
|
v
RESOLVE / CLOSE
|
+---- REOPEN ----> NEW TRIAGE CYCLE
` ` `

The API layer provides explicit contracts around each of these capabilities while keeping AI recommendations separate from authoritative business state.